Find the cheapest model that still does the job

In this guide, you'll run the same evaluator under a few model choices and keep the cheapest model that still catches the bad output. The sample task is a visual QA check that flags broken infographic tiles. The same pattern works for classifiers, extractors, routers, and other repeated steps where cost matters.

Setup

Load the launch helpers.

In [1]:
```
import pathlib
import sys


def _find_visual_artifact_example_root():
    cwd = pathlib.Path.cwd().resolve()
    candidates = []
    for base in (cwd, *cwd.parents):
        candidates.append(base)
        candidates.append(base / "examples" / "notebooks" / "visual_artifact
    for candidate in candidates:
        if (candidate / "shepherd_usecases" / "visual_artifact" / "launch.py
            return candidate
    raise RuntimeError(
        "Cannot find examples/notebooks/visual_artifact. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    )


example_root = _find_visual_artifact_example_root()
if str(example_root) not in sys.path:
    sys.path.insert(0, str(example_root))
try:
    from shepherd_usecases.visual_artifact import launch
    from shepherd_usecases.visual_artifact import viz
except Exception as exc:
    raise RuntimeError(
        "Could not import the visual-artifact notebook helpers. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    ) from exc

launch.bootstrap(example_root=example_root)
```

What this run does

Right-sizing runs the same evaluator under each model choice, grades every run against a deterministic oracle, then keeps the cheapest model that still caught the bad tile.

The input

The run starts with one prompt and a few model choices. `model_choices` maps each tier name to a model identifier; only the model changes between evaluator runs.

In [2]:
```
prompt = launch.default_prompt()
model_choices = launch.model_choices()
model_choices
```

Out[2]:
```
{'high': 'opus', 'mid': 'sonnet', 'cheap': 'haiku'}
```

Run Shepherd

Open a workspace for this comparison. Every evaluator run and the selector attach to it.

In [3]:
```
workspace = launch.open_workspace("model-right-sizing-lab", prompt=prompt, m
```

Run the evaluator once per model choice and grade the results. Each run goes through `launch.run_static` with a different model tier; `grade_runs` checks each verdict against a deterministic oracle — a check that already knows which tile is broken — and marks each run as passed or missed.

In [4]:
```
runs = {}
for config_name, model in model_choices.items():
    runs[config_name] = launch.run_static(
        workspace,
        name=f"rightsize-{config_name}",
        output_path=launch.VERDICT_PATH,
        output_content=launch.evaluator_content(config_name, model),
        model=model,
        metadata={"model_tier": config_name, "model": model},
    )

graded = launch.grade_runs(runs)
viz.show(viz.table([
    {
        "config": name,
        "model": item.model,
        "cost": item.cost,
        "catches hard fail": item.catches_hard_fail,
        "state": "passed" if item.passed else "missed",
    }
    for name, item in graded.items()
]))
```

config	model	cost	catches hard fail	state
high	opus	high	True	passed
mid	sonnet	medium	True	passed
cheap	haiku	low	False	missed

With every run graded, run the selector. It receives each run's verdict as an artifact ref in `verdict_refs` and reads them to pick the cheapest model that passed. `read_json` retrieves the selector's decision from `DECISION_PATH`.

In [5]:
```
verdict_refs = {
    f"verdict_{name}": launch.artifact_ref(item.run, launch.VERDICT_PATH, la
    for name, item in graded.items()
}
selector = launch.run_with_artifact_refs(
    workspace,
    name="selector",
    refs=verdict_refs,
    output_path=launch.DECISION_PATH,
    output_content=launch.selector_content(graded),
    after=[item.run for item in graded.values()],
)
decision = launch.read_json(selector, launch.DECISION_PATH)
decision
```

Out[5]:
```
{'dropped': ['high', 'cheap'], 'kept': 'mid', 'passed': ['high', 'mid']}
```

Mark the winning run as selected, discard the rest, and release the selector's output.

In [6]:
```
for name, item in graded.items():
    if name == decision["kept"]:
        item.run.output().select()
    else:
        item.run.output().discard()
selector.output().release()
```

Out[6]:
```
RetainedOutputSettlementResult(scope=ScopeInfo(name='run-a719dd1b1c07', ref
='refs/metagit/scopes/run-a719dd1b1c07', instance_id='91049a3895ef', creati
on_oid='4c2be637fd78ec9d9325de3cda2e06af951639a9', world_id='world_a4cdc369
06ca'), parent=ScopeInfo(name='ground', ref='refs/metagit/ground', instance
_id='ground-f72af3565361', creation_oid='', world_id='world_02f74456cfa5'),
output_world_oid='689cfd5c8a6c693b5680f9dda519b96dbe70e3ca', parent_world_b
efore='a6f053942367873b2994090a2bbe4a89bbab8fab', parent_world_after='a6f05
3942367873b2994090a2bbe4a89bbab8fab', settlement=RetainedOutputSettlement(s
cope_name='run-a719dd1b1c07', scope_ref='refs/metagit/scopes/run-a719dd1b1c
07', scope_instance_id='91049a3895ef', parent_ref='refs/metagit/ground', ha
ndoff_ref='refs/metagit/seals/b64u_cnVuLWE3MTlkZDFiMWMwNw/b64u_OTEwNDlhMzg5
NWVm', output_world_oid='689cfd5c8a6c693b5680f9dda519b96dbe70e3ca', binding
='workspace', store_id='store_workspace', resource_id='fs:repo-main', candi
date_id='primary', candidate_head='f1c950ef7eaebec1d4159191355725eb00322f4
c', action='released', operation_id='release_retained_5ad00376d491840a9dcaf
d30a7180174', parent_world_before='a6f053942367873b2994090a2bbe4a89bbab8fa
b', parent_world_after='a6f053942367873b2994090a2bbe4a89bbab8fab', settleme
nt_ref='refs/metagit/settlements/b64u_cnVuLWE3MTlkZDFiMWMwNw/b64u_OTEwNDlhM
zg5NWVm/b64u_d29ya3NwYWNl/b64u_cHJpbWFyeQ', p030_authority_operation_id=Non
e, p030_authority_settlement_operation_id=None, p030_authority_outcome=Non
e))
```

Trace

How does Shepherd run one task under many models and let a selector read them all? The trace below captures every action an agent took while executing the tasks above, plus the relationships among those actions. Click the nodes to learn more about the events.

In [7]:
```
viz.show(viz.trace(workspace.flow, runs | {"selector": selector}, height="62
```

Events

kind	run/flow	label	payload
flow.opened	flow-a9a1a8818dde		name='model-right-sizing-lab'
flow.fork.requested	run-f51ac7997f28	high	name='rightsize-high', after=[]
run.lifecycle	run-f51ac7997f28	high	task_id='shepherd_usecases.visua enforcement='advisory'
provider.invocation	run-f51ac7997f28	high	execution_id='task-execution-542 provider_event_kind='provider.in evidence_role='provider_provenan {'args_digest': 'sha256:f003b0bc 'task_id': 'shepherd_usecases.vi
provider.invocation	run-f51ac7997f28	high	execution_id='task-execution-542 provider_event_kind='provider.in evidence_role='provider_provenan {'artifact_path': 'verdict.json' 'sha256:49363d1608afe149a67eb5cc
run.output.published	run-f51ac7997f28	high	output_name='workspace', output_ binding='workspace', output_worl
run.output.settled	run-f51ac7997f28	high	output_name='workspace', output_ state='discarded', settlement_ref='refs/metagit/set
flow.fork.requested	run-f28062b582fa	mid	name='rightsize-mid', after=[]
run.lifecycle	run-f28062b582fa	mid	task_id='shepherd_usecases.visua enforcement='advisory'
provider.invocation	run-f28062b582fa	mid	execution_id='task-execution-2eb provider_event_kind='provider.in evidence_role='provider_provenan {'args_digest': 'sha256:cb97025b 'task_id': 'shepherd_usecases.vi

In [8]:
```
workspace.close()
```

Want to understand how it works?

If you want to understand how this works in greater detail, open the Model Right-Sizing internals notebook.

Next steps

- Find the best approach by trying several alternatives
- Recover a pipeline when one step drifts